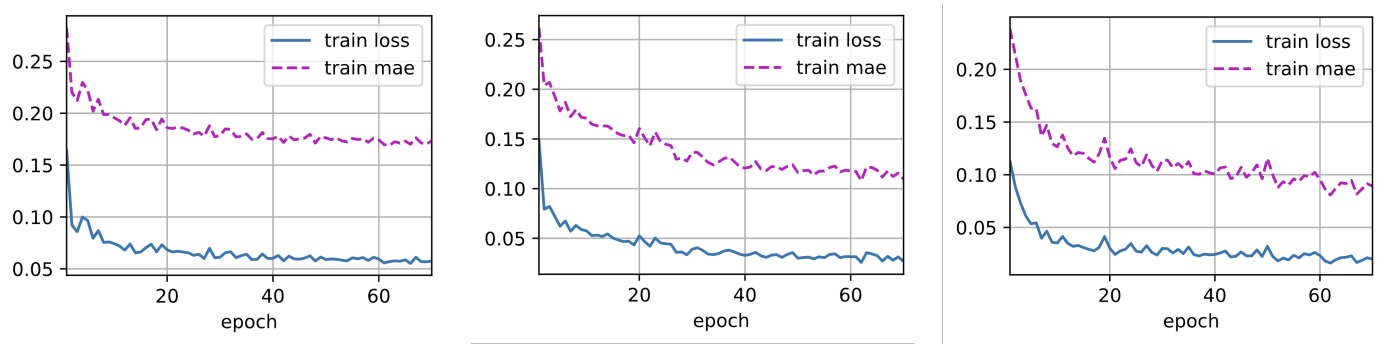
Latent Representation Study for Neural Curve Fitting

ECS659U: Neural Networks & Deep Learning

Mobin Mohaghegh

1 Experimental setup (Tasks 1 & 2)

I implemented the required encoder–decoder architecture as two jointly trained MLPs. The *encoder* receives each observed pair (x_o, y_o), maps it through two hidden layers of width $h_{\text{dim}} = 128$ with ReLU activations, and projects to a per-point feature $r_o \in \mathbb{R}^{r_{\text{dim}}}$. The set-level representation is the *average* over the observed set, $r_O = \frac{1}{N}\sum_o r_o$, which makes the encoder permutation-invariant. The *decoder* concatenates r_O with each target input x_t and passes the pair through two hidden layers of width 128 (ReLU), producing $\hat{y}_t \in \mathbb{R}$. Hidden layers use Kaiming-uniform initialisation (matched to ReLU); the final projection of each MLP uses Xavier-uniform. Across all runs, the training setup is identical except for r_{dim}: batch size 128, 70 epochs, MSE loss, `AdamW` with initial learning rate 2×10^{-3} and weight-decay 5×10^{-5}, and `ReduceLROnPlateau` with factor 0.8, patience 8, threshold 5×10^{-4}, cooldown 4, and minimum learning rate 10^{-6}. Each mini-batch samples a *random* number of observed points $N \sim \mathcal{U}\{5, \ldots, 20\}$ from the 100 available, matching the test-time observation protocol. All three models (`model2`, `model4`, `model8`, with $r_{\text{dim}} \in \{2, 4, 8\}$) are trained under identical settings, so any difference in performance reflects latent capacity rather than the optimisation schedule.



`model2`: final loss = 0.0575, final MAE = 0.1735 | `model4`: final loss = 0.0268, final MAE = 0.1094 | `model8`: final loss = 0.0200, final MAE = 0.0891

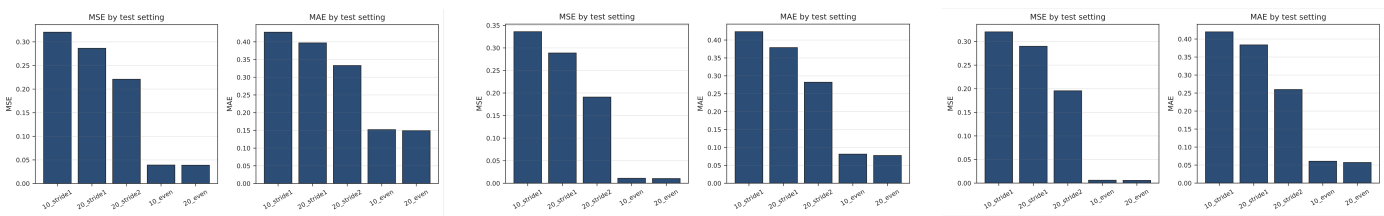
Figure 1: Per-epoch training loss/MAE for the three latent sizes under the same optimiser. Larger r_{dim} fits the training set more tightly, but this does *not* by itself determine the best test-time generalisation (see Tasks 3–4).

2 Task 3 – Model comparison

Each trained model is evaluated on the provided `test_data.csv` under the five observation settings supplied in `test_observations.pt`. Table 1 gives the MSE and MAE per setting, together with the overall averages and RMSE.

	MSE			MAE		
Setting	model2	model4	model8	model2	model4	model8
`10_stride1`	**0.3205**	0.3360	**0.3205**	0.4272	0.4234	**0.4199**
`20_stride1`	**0.2864**	0.2887	0.2899	0.3970	**0.3789**	0.3838
`20_stride2`	0.2208	**0.1910**	0.1956	0.3331	0.2822	**0.2598**
`10_even`	0.0392	0.0110	**0.0061**	0.1522	0.0813	**0.0607**
`20_even`	0.0388	0.0104	**0.0057**	0.1492	0.0775	**0.0572**
Overall avg.	0.1811	0.1674	**0.1636**	0.2917	0.2487	**0.2363**
Overall RMSE	0.4256	0.4092	**0.4044**			

Table 1: Per-setting and overall test errors for the three latent sizes. Best value in each row is in bold.



(a) `model2` ($r_{\text{dim}} = 2$) **(b)** `model4` ($r_{\text{dim}} = 4$) **(c)** `model8` ($r_{\text{dim}} = 8$)

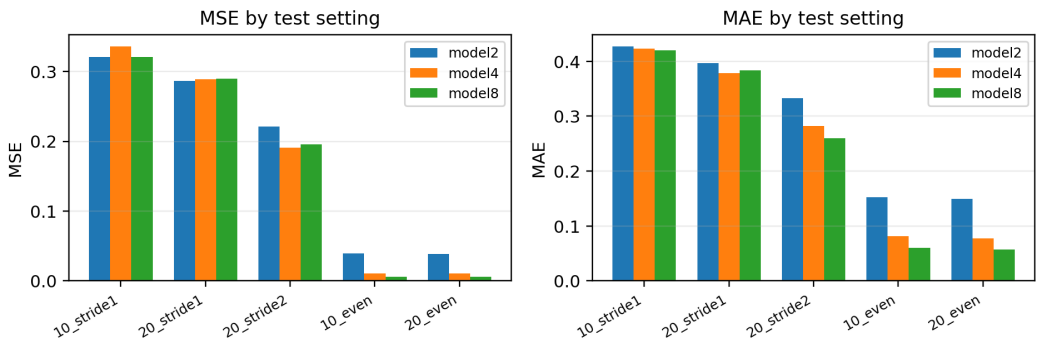


Figure 2: Per-setting MSE/MAE for each model, followed by a direct side-by-side comparison. All three remain close on the extrapolation-dominated `stride` settings, but they diverge sharply on the interpolation-friendly `even` settings.

3 Task 4 – Best latent size, most significant improvement, and data dimensionality

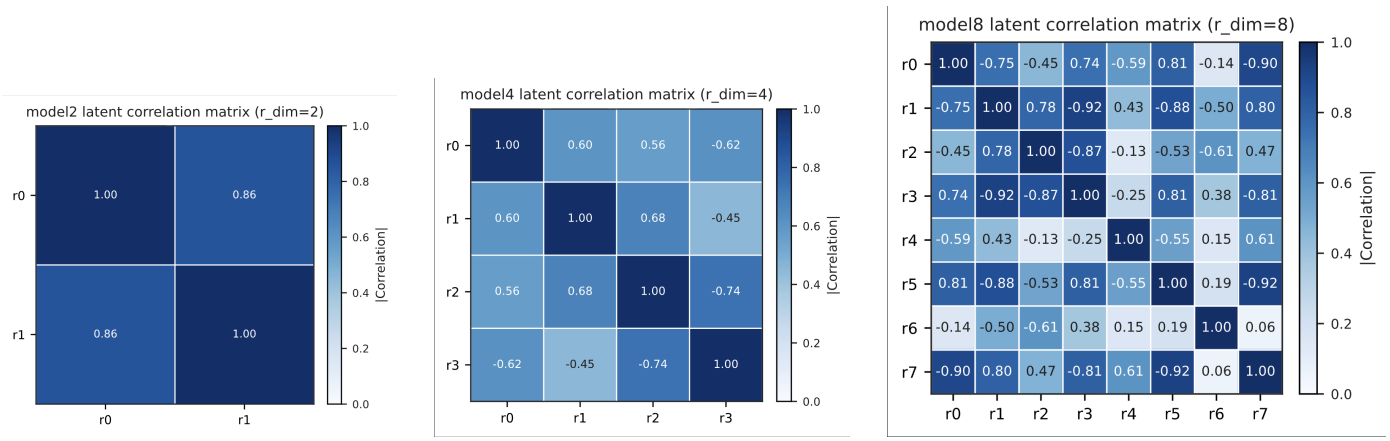
Best overall model. Judged by MSE and RMSE – the metrics best aligned with the MSE training objective – `model8` is the strongest model on this final run. It attains the lowest average MSE (0.1636), the lowest average MAE (0.2363), and the lowest RMSE (0.4044). `model4` remains very competitive and is only slightly behind on the overall metrics, but `model8` is now the best choice when the criterion is the final test performance.

Which latent size offered the most significant improvement? The largest jump is still from $r_{\text{dim}} = 2$ to $r_{\text{dim}} = 4$. Relative to `model2`, `model4` improves overall MSE by **7.6%**, MAE by **14.7%**, and RMSE by **3.9%**. The effect is concentrated in the *even* settings, where MSE drops by 71.9% (`10_even`: $0.0392 \to 0.0110$) and 73.2% (`20_even`: $0.0388 \to 0.0104$). The second step, $r_{\text{dim}} = 4 \to 8$, gives a smaller overall gain (MSE improves by only 2.3%) and a mixed per-setting picture – `model8` is better on the `even` settings and on the overall averages, but `model4` remains better on `20_stride2` in MSE (0.1910 vs. 0.1956). In short, the most meaningful capacity jump happens at $r_{\text{dim}} = 4$; moving to $r_{\text{dim}} = 8$ gives a smaller incremental improvement.

How many independent variables generated the data? A direct algebraic check on `train_data.csv` and `test_data.csv` confirms that every curve is a cubic polynomial of x. Regressing each sampled function onto the basis $\{x^3, x^2, x, 1\}$ produces a mean squared reconstruction error of $\sim 9.14 \times 10^{-30}$ on the training set and $\sim 8.65 \times 10^{-30}$ on the test set – numerical zero. Therefore the data-generating process has exactly **four independent variables**: the cubic coefficients (a, b, c, d) in $y = ax^3 + bx^2 + cx + d$. This explains the observed ranking: $r_{\text{dim}} = 2$ is *under-complete* (cannot represent a 4-dimensional family), $r_{\text{dim}} = 4$ *matches* the intrinsic dimensionality, and $r_{\text{dim}} = 8$ is *over-complete*, giving the model extra redundant directions that the same fixed-budget optimisation can only use partially.

4 Task 5 – Disentanglement of the most improved model ($r_{\text{dim}} = 4$)

Since `model4` offered the most meaningful improvement over the under-complete baseline, it is the natural target for the disentanglement analysis. If the encoder had learned a clean factored representation, each latent coordinate would capture one independent generating coefficient, and the off-diagonal correlations between latent dimensions would be close to zero. Figure 3 shows the correlation matrices for all three models and makes the comparison concrete.



(a) `model2` **(b)** `model4` **(c)** `model8`

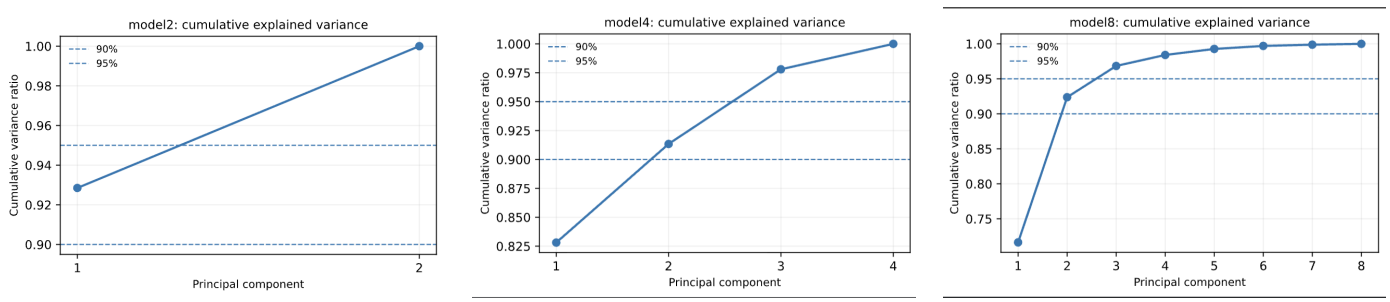
Figure 3: Pairwise correlations of the latent codes on the full test set. `model4` shows systematic off-diagonal coupling ($|\rho| \approx 0.45$–0.74), so its four coordinates are still entangled rather than axis-aligned with the four cubic coefficients.

Reading the model4 correlation matrix. Every off-diagonal entry in Fig. 3(b) has magnitude between 0.45 and 0.74. The signs show that r_0, r_1, and r_2 are positively correlated with one another, while r_3 is negatively correlated with all three. A truly disentangled representation would show near-zero off-diagonal structure; here the coupling is weaker than in the previous run, but it is still substantial. This means the four latent coordinates are *not* aligned with the four cubic coefficients – they behave as mixed directions rather than independent factors.

PCA quantifies the effective dimensionality. Applying PCA to the latent codes of each model gives:

Model	PC1 var. ratio	PC2 var. ratio	PC3 var. ratio	$n_{90\%}$	$n_{95\%}$	Effective rank
`model2`	0.9285	0.0715	–	1	2	1.2937
`model4`	0.8281	0.0853	0.0646	2	3	1.8720
`model8`	0.7163	0.2074	0.0447	2	3	2.3474

Table 2: PCA diagnostics on the latent codes. $n_{90\%}/n_{95\%}$ are the smallest numbers of components needed to explain $\geq 90\%/95\%$ of the variance; effective rank is $\exp(-\sum_i p_i \ln p_i)$ for normalised eigenvalues p_i.



(a) `model2` **(b)** `model4` **(c)** `model8`

Figure 4: Cumulative explained variance. `model4` needs two principal components to exceed 90% variance and three to exceed 95%, so it is less collapsed than `model2` but still far from four independent latent axes.

Interpretation. For `model4`, the first principal component accounts for 82.8% of the total latent variance, two components account for 91.3%, and three account for 97.8%; the effective rank is only ≈ 1.87 out of a nominal 4. In other words, although the latent has four coordinates, it behaves more like a low-dimensional mixed code than four disentangled axes, so it is *not* capable of cleanly separating the four independent cubic coefficients. The model has nevertheless learned to regress accurately because the decoder can compensate for the entanglement: it only needs a sufficient statistic of the observed set, not an axis-aligned factor code.

Comparison with the other two models. `model2` is the most collapsed representation: one principal component already explains 92.9% of the variance and the effective rank is only 1.29, which is consistent with an under-complete two-dimensional bottleneck. `model8` has the highest effective rank (2.35) and spreads variance across more principal directions, but several full-test correlations are still very strong (for example r_1–$r_3 \approx -0.93$, r_0–$r_7 \approx -0.91$, and r_5–$r_7 \approx -0.92$). Taken together, the three models show that increasing latent size improves task performance, but the standard sum-of-features bottleneck with MSE alone is insufficient to enforce a disentangled representation – stronger inductive biases would likely be needed to recover the four cubic coefficients as independent latent axes.